



Aluminum Casting Inspection Using Deep Learning: A Method Based on Convolutional Neural Networks

Domingo Mery¹

Received: 9 April 2019 / Accepted: 16 January 2020 / Published online: 21 January 2020
© Springer Science+Business Media, LLC, part of Springer Nature 2020

Abstract

Castings produced for the automotive industry are considered important components for overall roadworthiness. To ensure the safety of construction, it is necessary to check every part thoroughly using non-destructive testing. X-ray testing rapidly became the accepted way of controlling the quality of die-cast pieces. In this paper, we present a Convolutional Neural Network (CNN) for defect detection in castings. In order to train the CNN model, a large dataset is necessary. We build the dataset by using synthetic defects. They are simulated using 3D ellipsoidal models and Generative Adversarial Networks (GAN). We compare different portions of ellipsoidal/GAN defects in the training subset. In our experiments, the use of GAN defects does not play a relevant role in this solution. However, ellipsoidal defects helped to achieve better performance. Ellipsoidal defects from any size and orientation could be superimposed onto real X-ray images in any location. In addition, we tested several CNN configurations, the best one, that we call Xnet-II, has 30 layers and more than 1,350,000 parameters. It has been trained using a dataset with around 640,000 patches containing 50% of ellipsoidal defects and 50% of real background captured from different casting types. The model was tested using sliding-windows methodology on whole X-ray images achieving promising results (mPA = 0.7102): the model was able to detect real defects from different casting types. We believe that the methodology presented could be used in similar projects that have to deal with automated detection of defects.

Keywords X-ray testing · Castings · Automated inspection · Computer vision · Deep learning

1 Introduction

Light-alloy castings produced for the automotive industry, such as wheel rims, steering knuckles, and steering gear boxes are considered important components for overall roadworthiness. Non-homogeneous regions can be formed within the workpiece in the production process. These are manifested, for example, by bubble-shaped voids, fractures, inclusions or slag formation. To ensure the safety of construction, it is necessary to check every part thoroughly using X-ray testing. In casting inspection, automated X-ray systems have not only raised quality, through repeated objective inspections and improved processes but have also increased productivity and consistency by reducing labor costs. Some examples are illustrated in Fig. 1.

X-Ray testing is one of the more acceptable ways for examining an object without destroying it [1]. The purpose of this non-destructive method is to detect or recognize certain parts of interest that are located inside a test object and are thus not detectable to the naked eye. The material defects occurring in the casting process such as cavity, gas, inclusion, and sponge must be detected to satisfy the security requirements; consequently, it is necessary to check 100% of the parts.

The principal aspects of an X-ray testing system comprise the following steps: (i) The test object is located in the desired position using a manipulator. (ii) The X-ray source generates X-rays which pass through the test object. (iii) The X-rays are detected and converted by a flat panel in order to obtain a digital X-ray image. (iv) Computer vision algorithms are used to evaluate the X-ray image. Sensors, like flat detectors made of amorphous silicon, have been widely used in some industrial inspection systems [2]. In these detectors, the energy from the X-ray is converted directly into an electrical signal by a semiconductor. Different methods for the automated detection of casting discontinuities using computer vision have

✉ Domingo Mery
domingo.mery@uc.cl
https://domingomery.ing.puc.cl

¹ Department of Computer Science, Pontificia Universidad Católica de Chile, Santiago, Chile

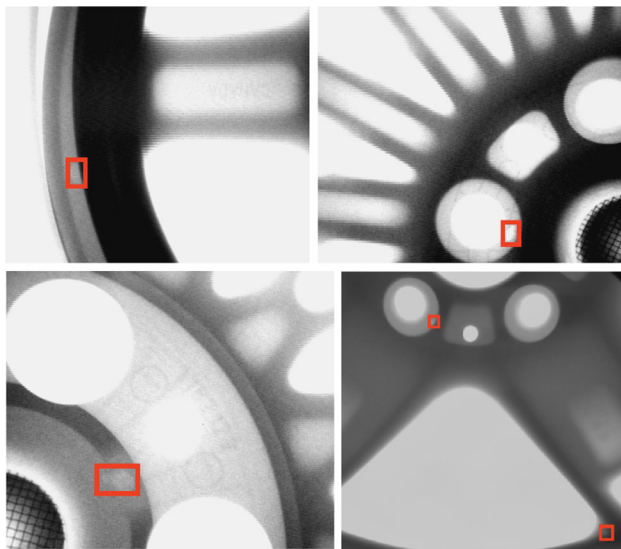


Fig. 1 Real defects in X-ray images of wheels

been described in the literature over the past thirty years [1]. One can see that the different approaches for this task can be divided into three groups [3,4]:

- **Reference methods:** In reference methods, it is necessary to take still images at selected programmed inspection positions. A test image is then compared with the reference image. If a significant difference is identified, the test piece is classified as defective.

- **Methods without apriori knowledge of the structure:** These approaches using pattern recognition, expert systems, artificial neural networks, general filters or multiple views analyzes to make them independent of the position and structure of the test piece.
- **Computed tomography:** These approaches use computed tomography to make a reconstruction of the cast piece and thereby detect discontinuities.

A selection of recent approaches are summarized in Table 1. In general, they use classical image processing and pattern recognition techniques with a single view or multiples views, where the decision is made by analyzing the correspondence among the views. There are some approaches that use deep learning methodologies as well.

In recent years, deep learning has been successfully used in image and video recognition (see for example [22–24]), and it has been established as the state of the art in many areas of computer vision. The key idea of deep learning is to replace *handcrafted* features with features that are *learned* efficiently using a hierarchical feature extraction approach. There are several deep architectures such as deep neural networks, convolutional neural networks, Boltzmann machines, energy-based models, and deep belief networks, among others [23]. Convolutional neural networks, CNN, originally inspired by a biological model [25], is a very powerful class of methods for image recognition [26] which replaces feature extraction and classification with a single neural network.

In our previous work [14], we designed a CNN network, called Xnet, with 10 layers. In the design, we explored sev-

Table 1 Defect recognition on castings

Authors	References	Year	Description
Carrasco and Mery	[5]	2011	Multiple view correspondence with non-calibrated model
Cogranne and Retraint	[6]	2014	Statistical hypothesis testing using nonparametric tests
Ferguson et al.	[7]	2017	Deep learning: Faster R-CNN architecture
Kamalakannan et al.	[8]	2017	Image processing based on spatial based segmentation
Li et al.	[9]	2006	Wavelet technique
Li et al.	[10]	2015	Peak location algorithm combined with neural networks
Lin et al.	[11]	2018	Inter-frame deep convolution neural network strategy
Mery and Filbert	[12]	2002	Multiple view model with calibrated model
Mery	[13]	2015	Multiple views using an un-calibrated tracking approach
Mery and Arteta	[14]	2017	Comparative evaluation of 24 computer vision methods
Pieringer and Mery	[15]	2010	3D model from 2D images
Pizarro et al.	[16]	2008	Multiple views based on affine transformation
Ramirez and Allende	[17]	2013	Generative and discriminative approaches
Tang et al.	[18]	2009	Segmentation using fuzzy model
Zhang et al.	[19]	2018	Image processing based on adaptive thresholding
Zhao et al.	[20]	2014	Statistical feature based on grayscale arranging pairs
Zhao et al.	[21]	2015	Sparse representations

eral CNN models for our task of defect detection. First, we took standard CNN architectures that have been previously trained on the ImageNet dataset [27], a large and highly-variable collection of (optical) images, and used them as general-purpose feature extractors. Training CNN models from scratch with our own dataset did not yield good results, the obtained accuracy of our preliminary experiments was around 85%. The models were found to be strongly overfitted on the training data. This can be attributed to the small size of the used dataset in [14]. In order to overcome this problem, in this paper, we propose to use simulated data. Thus, we use two approaches to generate synthetic patches with defects: simulation using ellipsoidal models and simulation using Generative Adversarial Networks (GAN). In our work, we design a new model, called Xnet-II, with 30 layers. In addition, we tested our proposed CNN using a sliding-windows approach on whole X-ray images.

2 Method

2.1 Overview

Our method follows a sliding-window methodology [1] as illustrated in Fig. 2. Thus, a detection window (see the orange square in Fig. 2a) is slid over an input image in both horizontal and vertical directions, and for each localization of the detection window, a classifier decides to which class belongs the corresponding portion of the image according to its representation. In our approach, the classifier is a Convolutional Neural Network (CNN) that is used to identify one of two classes: ‘defects’ or ‘no-defects’. For this end, a huge number of patches of each class is used to train the CNN model. The patches, that have the same size of the detection window, can contain a defect (for the ‘defect’ class) or not (for the ‘no-defect’ class) as shown in Fig. 3. Finally, the locations of the X-ray image that have been detected as ‘defects’ (see green dots in Fig. 2b) are analyzed using image processing. Thus,

we can determine which regions of the image are defects or not (see detected regions in Fig. 2c).

In Sect. 2.2, we explain the CNN that has been designed for this problem. In order to train the CNN model, a huge number of both classes are required. Typically, however, the number of defects in the datasets is very small. In Sect. 2.3, we address the problem of unbalanced data using data augmentation. Two methods are used to increase the number of patches that belong to class ‘defect’. Finally, in Sect. 2.4, we explain the image processing methods that are used to detect the ‘defect’ regions.

2.2 Convolutional Neural Network (CNN)

The proposed method for patch classification is a convolutional neural network (CNN) of eight blocks as illustrated in Fig. 4. It is based on a basic feed-forward (sequential) network [28]. In this Section, we explain the network architecture.

In our case, the input image corresponds to a patch, i.e., a cropped image of 32×32 pixels that might have two classes: ‘defects’ or ‘no-defects’ as illustrated in Fig. 3. Our CNN maps an input image $\mathbf{X}$ onto an output vector $\mathbf{y} = \mathcal{G}_L(\mathbf{X})$, where the function $\mathcal{G}_L$ can be viewed as a feed-forward network with L layers which are typically linear operations followed by non-linearities (e.g., convolutions followed by rectified linear units [29]). These layers f_l , for $l = 1 \dots L$ contain parameters $\mathbf{w} = (\mathbf{w}_1, \dots, \mathbf{w}_L)$ that can be discriminatively learned from training data: a set of input images $\mathbf{X}_i$ and their corresponding labels $\mathbf{y}_i$, for $i = 1, \dots, n$, so that $\sum_i \ell(\mathcal{G}_L(\mathbf{X}_i), \mathbf{y}_i) / n \rightarrow \min$, where ℓ is a loss function. This optimization problem can be solved using the back-propagation algorithm [30].

We tested several CNN configurations (see Sect. 3), and the best one—that we call Xnet-II—is explained in this section. Our own CNN, Xnet-II, for the defect detection, is an eight-block net as shown in Fig. 4 and Table 2. For this end, we use five ‘conv’ (convolutional) blocks and three ‘fc’ (fully con-

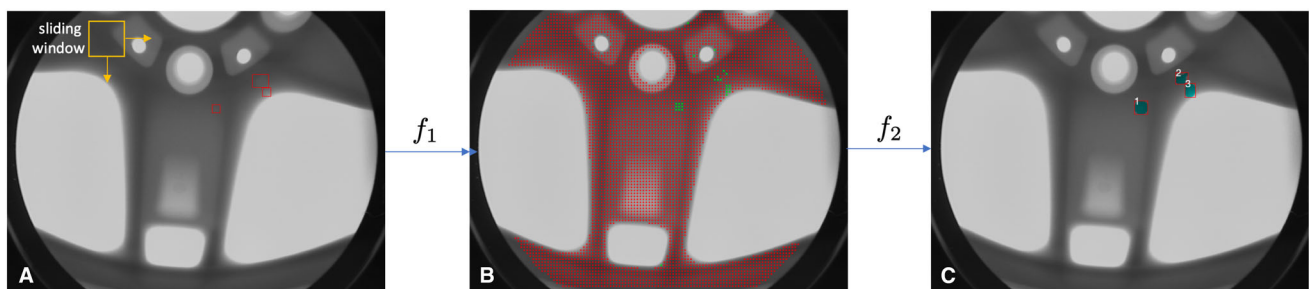


Fig. 2 Overview of the proposed method: The original X-ray image ‘A’ with three defects (see red rectangles) is processed by f_1 using a sliding-window approach. For each position of the detection windows (orange square) a patch is extracted and classified as ‘defect’ or ‘no-defect’ by a trained CNN. The dots of image ‘B’ show in red and green

the center of the patches that were classified as ‘defect’ and ‘no-defect’ respectively. Using image processing approaches the dots of image ‘B’ are processed by f_2 to detect the ‘defect’ regions. In this example, all defects were correctly detected with no false alarm

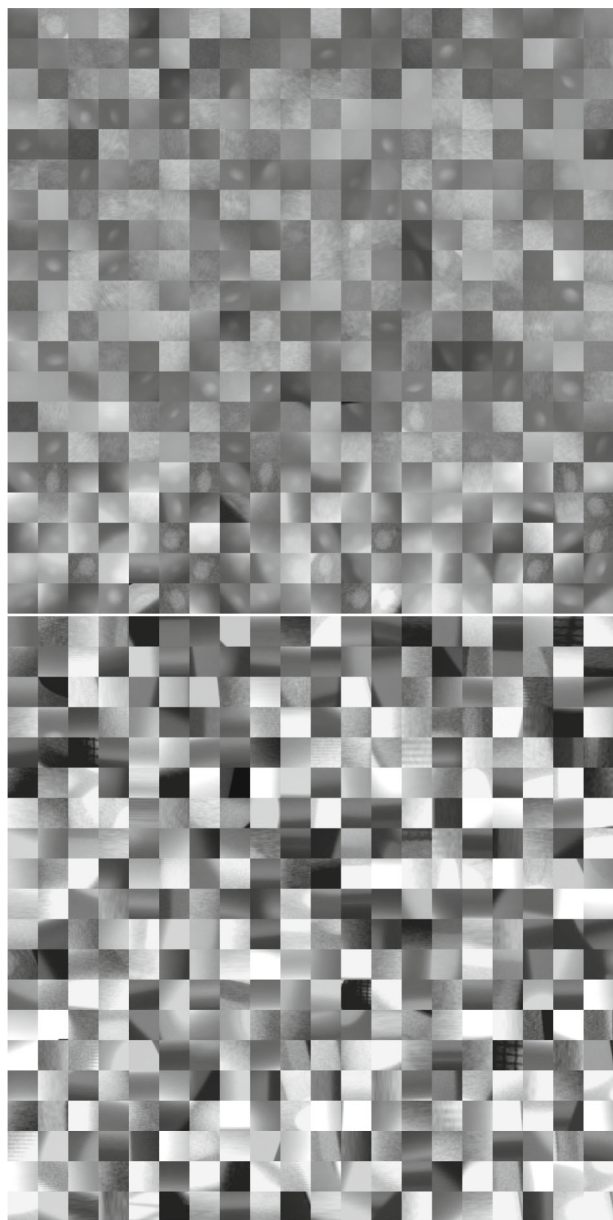


Fig. 3 Some ‘defects’ (top) and ‘no-defects’ (bottom) of our database. It is clear that there are some patterns that can be easily detected (see for example defects that are bright bubbles with dark background and no-defects that are regular structures with edges), however, the recognition of both classes can be very difficult for low contrast defects because they are very similar to homogenous no-defects

nected) blocks. In our notation, ‘conv’ block corresponds to a 2D convolution layer with batch normalization, max-pooling, and dropout layers. The dropout layer randomly turns off connections of the neural network during training, which has been shown to reduce significantly the over-fitting [31]. The batch normalization layer is used to achieve higher learning rates [32]. In each ‘conv’ block, the max-pooling layer reduces the size of the input image by half in each dimension (e.g., from 32×32 to 16×16 pixels in the first

block), and the 2D convolution layer increases the number of channels (e.g., from 20 to 40 in the second block) performing a linear convolutional operation as illustrated in Fig. 4. On the other hand, ‘fc’ block performs a fully connected operation in which all inputs are connected to all outputs like a multi-layer perceptron. The last block has two outputs, for two classes: ‘defects’ and ‘no-defects’. The details of all layers are shown in Table 2. In our CNN, we have 30 layers and approx. 1,350,000 parameters.

As we can see from Fig. 4 and Table 2, in our proposed architecture Xnet-II, the five ‘conv’ blocks are defined by the size of the 2D filter masks $\mathbf{p} = (7, 5, 3, 3, 3)$ and the number of channels $\mathbf{d} = (20, 40, 80, 160, 640)$. In this definition, the size of the kernel of ‘conv’ block i is $p_i \times p_i \times d_i$. On the other hand, the three ‘fc’ blocks are defined by $\mathbf{f} = (320, 160, 2)$, where f_i is the size of the output layer of ‘fc’ block i . In the experiments (see Sect. 3.2), we report the performance of different $(\mathbf{p}, \mathbf{d}, \mathbf{f})$ architectures.

2.3 Data Augmentation

In casting inspection (and other defect detection problems), usually, the available samples for training purposes are unbalanced: there are a few numbers of ‘defects’ and a huge number of ‘no-defects’ [33]. Furthermore, when training a CNN model it is necessary to have a large number of samples of every class. In order to overcome this problem, we propose to use simulated data for the class ‘defects’. Thus, we can have both (i) a balanced distribution of classes ‘defects’ and ‘no-defects’ and (ii) a huge amount of samples of each class. In this Section, we explain two approaches that can be used to generate synthetic patches with defects: simulation using ellipsoidal models (proposed in [34]) and simulation using Generative Adversarial Networks (GAN) (adapted from [35]).

2.3.1 Simulation of Ellipsoidal Defects

As mentioned before, in our work, we use simulated ‘defects’ to increase the number of samples of class defects in the training data. Typically, X-ray imaging can be modeled using the absorption law which characterizes the intensity distribution of X-rays through matter [36]:

$$\varphi(x) = \varphi_0 e^{-\mu x} \quad (1)$$

with μ absorption coefficient, x thickness of the irradiated matter, φ_0 incident energy flux density, and φ energy flux density after passage through matter with the thickness of x . In an X-ray digital image, the gray-value of a pixel can be linearly modeled as [37]:

$$I(x) = A \cdot \varphi(x) + B, \quad (2)$$

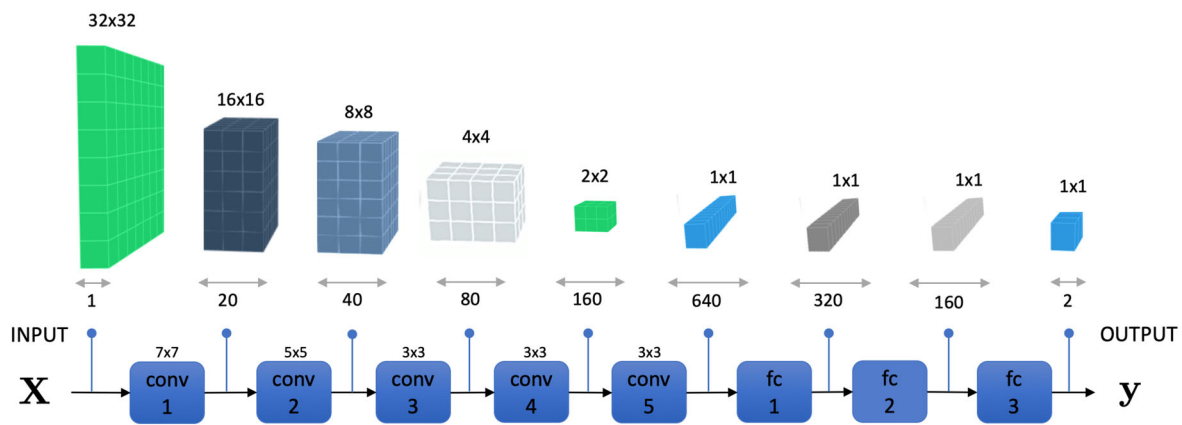


Fig. 4 Proposed network Xnet-II (see details in Table 2)

where A and B are constant parameters of the model that can be easily estimated [37]. If the material is a bubble of thickness d (with null absorption coefficient), the output energy flux density can be written as: $\varphi(x-d) = \varphi_0 \exp(-\mu(x-d))$. Thus, the new X-ray image can be modeled from (1) and (2) by:

$$I(x-d) = A \cdot \varphi(x-d) + B = (I(x) - B)e^{\mu d} + B. \quad (3)$$

Following this equation, it is possible to model the X-ray image of a casting with a simulated defect ' $I(x-d)$ ' from the X-ray image of casting with no defect ' $I(x)$ ' and a 3D model of the defect [37]. The approach simulates only the defect and not the whole X-ray image of the casting. A 3D defect can be modeled as an ellipsoidal cavity, which is projected and superimposed onto real X-ray image of a homogeneous casting with absorption coefficient μ . The new gray-value of a pixel, where the 3D defect is projected, depends only on four parameters: a) the original gray-value ' $I(x)$ ', b) the linear absorption coefficient of the examined material ' μ ', c) parameter ' B ', and d) the length ' d ' of the intersection of the 3D flaw with the modeled x-ray beam, that is projected into the pixel (see Fig. 5). A simulation of an ellipsoidal defect of any size and orientation can be done in any position of the casting. See more details in [34,37].

2.3.2 Simulation of Defects Using Generative Adversarial Networks (GAN)

Generative Adversarial Networks (GAN) have been used successfully in the last years to generate realistic synthetic data [38]. In our work, we use GAN to simulate defects, that are used as data augmentation in training data to increase the number of samples of 'defect' class. The key-idea of GAN is simple (see Fig. 6): a generator and a discriminator working together. The generator will be used to produce a synthetic defect from a noise source, whereas the discriminator will

be in charge to determine if the generated defect and the original one are real or fake. Both generator and discriminator are trained following a zero-sum game schema [39]. In our work, we use Deep Convolutional Generative Adversarial Networks (DCGAN) [35], where both generator and discriminator are sequential models [35] (see details of the implementation in Sect. 3.1.2).

2.4 Detection

As mentioned in Sect. 2.1, in our proposed method an X-ray image of casting is processed using a sliding-window approach (see Fig. 2a). For each position of the detection window, the corresponding patch is classified as 'defect' or 'no-defect' by the trained CNN (see red and green dots in Fig. 2b). In this section, we explain function f_2 of Fig. 2, that analyzes the detected 'defects' (green dots Fig. 2b) in order to detect regions of the image that have defects.

The detection approach follows a morphological strategy. The detected 'defects' are represented as pixels equals to '1' in a binary image that has the same size as the input X-ray image (the rest of the image has pixels equals to '0'). This binary image is dilated using a circle structure. In this new image, those pixels that do not belong to the test object are removed. Finally, the area of the reining regions are computed and those regions that are large enough are detected. The idea is to select those locations that are not isolated (e.g., only one location detected as 'defect' with no other detection in the neighborhood will be considered as noise). An example is shown in Fig. 2c.

3 Experimental Results

In this Section, we present the experiments and the results that we obtained using the proposed method. The Section is subdivided into four main parts. The first part, Sect. 3.1, is

Table 2 Proposed convolutional neural network (see Fig. 4)

Block	Layer (type)	Output shape	Parameters
→	input	$1 \times 32 \times 32$	0
conv-1	Conv2D (7×7)	$20 \times 32 \times 32$	1000
	Batch Normalization	$20 \times 32 \times 32$	128
	MaxPooling2D	$20 \times 16 \times 16$	0
	Dropout	$20 \times 16 \times 16$	0
conv-2	Conv2D (5×5)	$40 \times 16 \times 16$	20,040
	Batch Normalization	$40 \times 16 \times 16$	64
	MaxPooling2D	$40 \times 8 \times 8$	0
	Dropout	$40 \times 8 \times 8$	0
conv-3	Conv2D (3×3)	$80 \times 8 \times 8$	288,80
	Batch Normalization	$80 \times 8 \times 8$	32
	MaxPooling2D	$80 \times 4 \times 4$	0
	Dropout	$80 \times 4 \times 4$	0
conv-4	Conv2D (3×3)	$160 \times 4 \times 4$	115,360
	Batch Normalization	$160 \times 4 \times 4$	16
	MaxPooling2D	$160 \times 2 \times 2$	0
	Dropout	$160 \times 2 \times 2$	0
conv-5	Conv2D (3×3)	$640 \times 2 \times 2$	922,240
	Batch Normalization	$640 \times 2 \times 2$	8
	MaxPooling2D	$640 \times 1 \times 1$	0
	Dropout	$640 \times 1 \times 1$	0
fc-1	Dense	320	204,800
	Batch Normalization	320	1280
	Activation	320	0
	Dropout	320	0
fc-2	Dense	160	51,200
	Batch Normalization	160	640
	Activation	160	0
	Dropout	160	0
fc-3	Dense	2	322
	Activation	2	0
→	output	2	0
	Total parameters		1,346,010
	Trainable parameters		1,344,926
	Non-trainable parameters		1084

dedicated to the simulation of defects. The second part, Sect. 3.2, shows the different CNN architectures that we tested. The third part, Sect. 3.3, presents the detection results in whole X-ray images. Finally, Sect. 3.4 gives a discussion of our results.

In all experiments, we used GDXray dataset [40]. The X-ray images included in [40] can be used free of charge, for research and educational purposes only. It contains 2,727 X-ray images of castings. In our experiments, we used seven series C0001, C0010, C0015, C0021, C0031, C0034 and C0061, because they have an annotated ground truth that includes real defects. In each series, there is a unique cast-

ing piece that is radiographed from different points of view. Totally, there are 288 X-ray images (in average 41.1 X-ray images per casting piece).

3.1 Simulation of Defects

In this Section, we present the results of simulation of defects using the ellipsoidal and the GAN models as explained in Sect. 2.3 for data augmentation.

Fig. 5 Geometric model of the simulated ellipsoidal defect

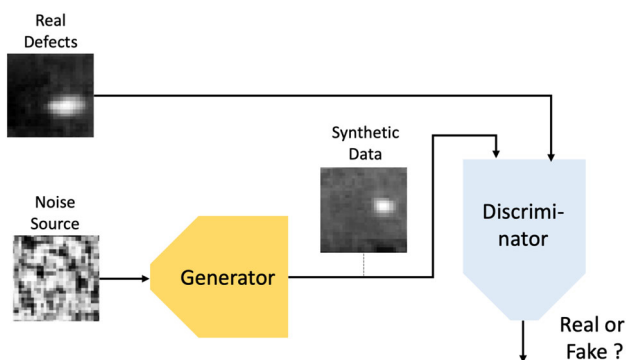
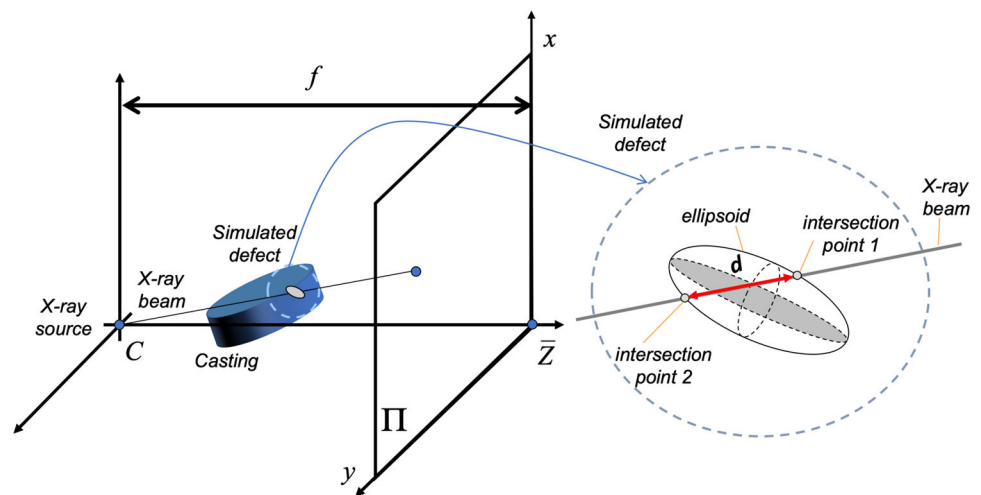


Fig. 6 GAN model for simulation of defects

3.1.1 Simulation of Ellipsoidal Defects

In order to simulate defects using the ellipsoidal model explained in Sect. 2.3.1, we use the command `Xsimdefect` of Matlab Toolbox `Xvis` implemented in [1].¹ The input variables of this command are the size of the three axes of the ellipsoid, the location, and orientation in 3D space of the ellipsoid, the linear absorption coefficient μ of the casting and a parameter called $x_{\max}$ defined as the maximal thickness where gray values are minimal (it is used to compute parameter B in (3)). We used the same configuration reported in [33]. Some examples are shown in Fig. 7. We can see that the defects have an elliptical shape of different sizes, and they are located and orientated in different ways. The size of the axes varies randomly from 1 to 6mm. In these examples, we show the whole image and the simulated defects. For training purposes, we use patches that contain the defects (not the whole image).

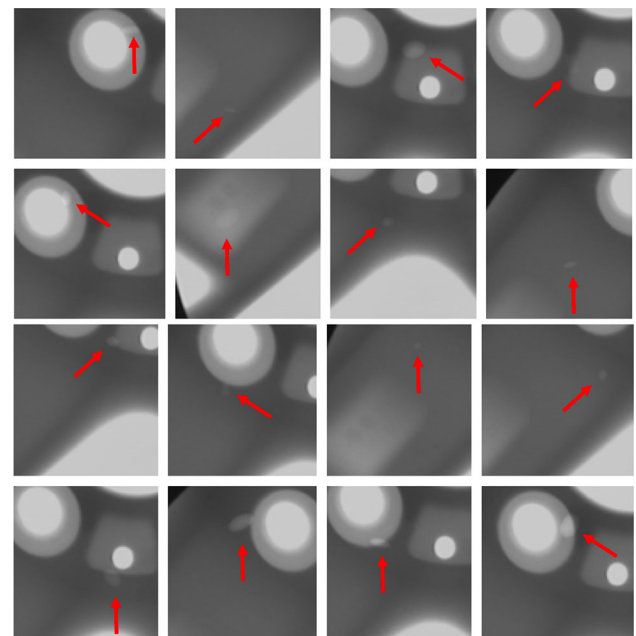


Fig. 7 Simulated defects using ellipsoidal model (see red arrows)

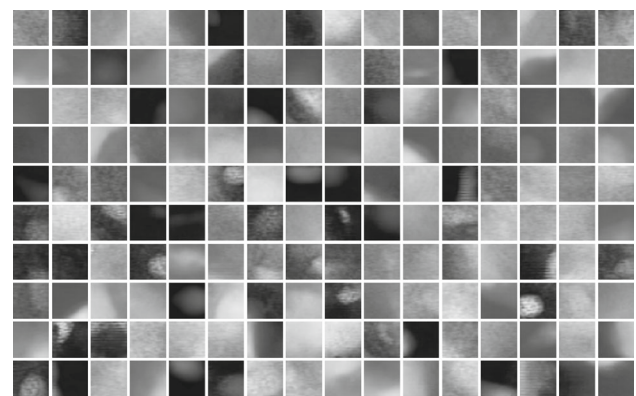


Fig. 8 Simulated patches with defects using GAN

¹ See <https://github.com/domingomery/Xvis>.

Table 3 Accuracy on training and testing dataset for different CNN architectures

	#	d	p	f	Training	Testing
	0	(7,5,3)	(32,64,128)	(256,64,2)	0.9618	0.8254
	1	(7,5,3)	(20,40,80)	(200,2)	0.9538	0.7145
	2	(7,5,3,3)	(20,40,80,160)	(320,160,2)	0.9748	0.9494
	3	(7,5,3,3)	(40,80,120,160)	(320,160,2)	0.9875	0.9613
	4	(7,5,3,3)	(40,80,120,160)	(320,640,2)	0.9550	0.9211
	5	(7,5,3,3,3)	(20,40,80,160,320)	(200,400,2)	0.9487	0.9124
→	6	(7,5,3,3,3)	(20,40,80,160,640)	(320,160,2)	0.9954	0.9687
	7	(7,5,3,3,3)	(20,40,80,160,640)	(100,200,400,2)	0.9938	0.9086

Table 4 Performance using different configurations of simulated defects

#	Training dataset	Ellipsoidal defects (%)	GAN defects (%)	Training accuracy	Validation accuracy	mPA (sliding-windows)
1	E100/G000*	100	0	0.8815	0.8745	0.6905
2	E100/G000	100	0	0.8869	0.8784	0.7102
3	E075/G025	75	25	0.8926	0.8881	0.6083
4	E050/G050	50	50	0.9265	0.9224	0.7063
5	E025/G075	25	75	0.9541	0.9525	0.6497
6	E000/G100	25	75	0.9999	1.0000	0.5000

3.1.2 Simulation Defects Using GAN

In order to generate synthetic defects using GAN approach as explained in Sect. 2.3.2, we use Deep Convolutional Generative Adversarial Networks (DCGAN) [35]. We use the open source implementation² that has been already tested in other applications such as faces, bedrooms, and handwritten digits. In this implementation, both generator and discriminator are sequential models (with 12 and 18 layers respectively). For the training of the GAN, we used real defects in different orientations and locations. The training step stopped after 150,000 iterations. Some results are shown in Fig. 8.

3.2 Selection of the CNN Architecture

In order to select the best CNN architecture for this problem, we trained CNN's with more than 20 different configurations of layers. The best ones are reported in Table 3. In this table, we use the nomenclature mentioned in Sect. 2.2 with n 'conv' (convolutional) blocks and m 'fc' (fully connected) blocks. 'conv' blocks are defined by n -element vectors $\mathbf{p}$ and $\mathbf{d}$, where the size of the kernel of 'conv' block i is $p_i \times p_i \times d_i$, for $i = 1 \dots n$. On the other hand, 'fc' blocks are defined by m -element vector $\mathbf{f}$, where f_i is the size of the output layer of 'fc' block i , for $i = 1 \dots m$. In our experiments, the best configuration was achieved by $\mathbf{p} = (7, 5, 3, 3, 3)$, $\mathbf{d} = (20, 40, 80, 160, 640)$ and $\mathbf{f} = (320, 160, 2)$, as we can

see in row # 6 of Table 3, where the accuracy on the training and testing dataset were 0.9954 and 0.9687 respectively.

We tested several sizes for the patch, from 18×18 to 40×40 pixels. The best results were obtained when using patches of 32×32 pixels. Thus, the datasets used in these experiments consists of patches of 32×32 pixels (defects and no-defects). The patches that belong to class 'defect' can be real or synthetic (using the simulated ellipsoidal defects and defects generated by GAN model reported in Sect. 3.1). The real patches are extracted in random locations of X-ray images of castings: for class 'no-defect' the area covered by the patch does not have any real defect, whereas for class 'defect' the patches contain at least one third of a real defect, i.e., a real defect can be present in more than one patch. Training and testing subsets are balanced: 50% of 'defects' and 50% of 'no-defects'. They are defined as follows:

- Training subsets consist of patches extracted from series C0010, C0015, C0021, C0031, C0034 and C0061 of GDXray. The 'defect' class was augmented using simulated ellipsoidal defects and defects generated by GAN model. We tried different portions of real/ellipsoidal/GAN 'defects', in all cases in the training subset there were 75,000 samples of 'defect' class and 75,000 samples of 'no-defect' class.
- Testing subset consists of patches of 32×32 pixels (50% of 'defects' and 50% of 'no-defects') extracted from series C0001 of GDXray. In the testing subset, all patches are real, i.e., there is no simulated patch for the class 'defect'. In the testing subset, there were 6,000 sam-

² See <https://github.com/eriklindernoren/Keras-GAN/tree/master/dcgan>.

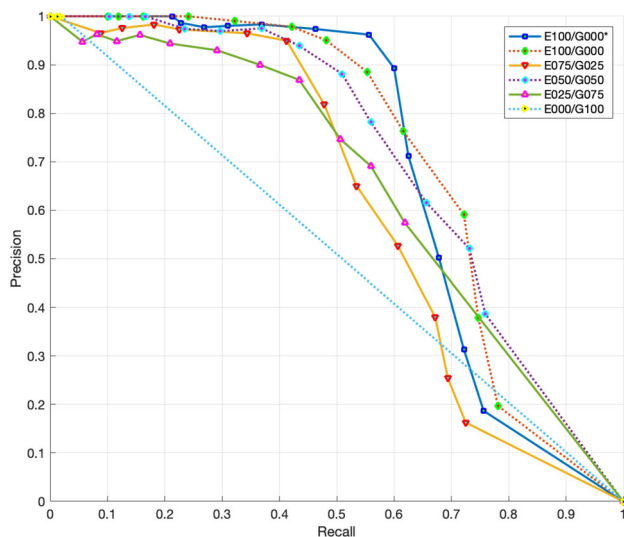


Fig. 9 Precision-Recall curves for different strategies

ples of ‘defect’ class and 6,000 samples of ‘no-defect’ class.

3.3 Detection with Sliding-Windows

We noticed that the performance of our CNN model, that was trained with the datasets mentioned in Sect. 3.2, was not good enough when testing it on whole X-ray images (using sliding-window approach). For this reason, we trained our CNN model in a larger dataset using the architecture previously selected (see details in Table 2 and performance in row #6 in Table 3). Thus, we build a new dataset with more than 500,000 patches for training purposes using the X-ray images of the seven casting series mentioned before as follows. The X-ray images of the series are enumerated with a sequential ID number. We select all images with the odd ID number, i.e., 147 X-ray images. For each selected X-ray image, we segmented the portion of the image where is the casting piece and remove those parts where the ground truth information indicates that there are defects. In these defects-free regions of interest, (i) we extract random patches for ‘no-defect’ class and (ii) we simulate ellipsoidal defects and extract from them patches for ‘defect’ class. The extraction of patches and the simulation of ellipsoidal patches were performed by the method explained in Sect. 3.2. In each image, we extract on average around 2200 patches for ‘defect’ class and 2200 patches for ‘no-defect’ class. Thus, we have a dataset of around 640,000 patches (50% for ‘defects’ and 50% for ‘no-defects’). Using stratified sampling, we select 80% of the samples for training (around 512,000 samples) and 20% for validation (around 128,000 samples). We call this dataset ‘E100/ G000’ because ‘defect’

class was built using 100% of simulated ellipsoids and 0% of defects generated by GAN model. We built variations of this dataset replacing a portion of ellipsoidal defects by GAN defects. We call these variations ‘ExGy’, where x% and y% are ellipsoidal and GAN defects respectively (with $x + y = 100\%$). The accuracy obtained on training and validation subsets is reported in Table 4. In addition, we define subset ‘E100/G000- i ’, for $i = 1 \dots 7$, in which we selected only those patches of ‘E100/G000’ that belong to series i . Again 80% and 20% of the samples were used for training and validation respectively. The average of the obtained performance in this experiment is reported in Table 4 as ‘E100/G000*’. The idea of the last experiment was to compare ad-hoc training versus general training. In an ad-hoc training, we use information of a specific casting type, that means the trained model used only X-ray images of this casting type, i.e., for another casting type we required another model trained specifically for this type. On the other hand, in the general model, the training and validation have been performed using only one general model that has the information of all seven casting types, i.e., only one model trained for all casting types.

In order to test the trained models on whole X-ray images, we follow the sliding-windows method explained in Sect. 2.4. In this experiment, for testing purposes, we used all X-ray images of the seven series with even ID number. The defects of the testing X-ray images are all real defects (we did not simulate anyone). It is worth mentioning, that X-ray images used in the testing dataset were not used in training/validation datasets, and the real defects that are in the testing X-ray images are not present in the X-ray images of training/validation datasets. Using the ‘Intersection over Union’ criteria (IoU) that measures how much the real and detected defects overlap [41], we measure the precision and recall values. The precision-recall curve is computed by varying a detection parameter θ . For this end, we define that our CNN model will detect a ‘defect’ if $y_1 - y_0 > \theta$, where y_1 and y_0 are the two activation outputs of our model (one for class ‘defect’ and another for class ‘no-defect’) and parameter θ is a number that can take values between 0 and 1. The obtained precision-recall curves are illustrated in Fig. 9. We measure the performance metric called mean Average Precision (mAP), defined as the average of the maximum precisions at different recall values. The obtained mAP values are reported in the last column of Table 4. We observe, that model ‘E000/G100’ that had an outstanding performance in the training/validation subsets could not detect any real defect of the testing X-ray images, for this reason, the performance is very poor (mPA=0.5). The best performance was achieved by ‘E100/G000’. Results using this model on a whole X-ray image of each series are shown in Figs. 2 and 10.

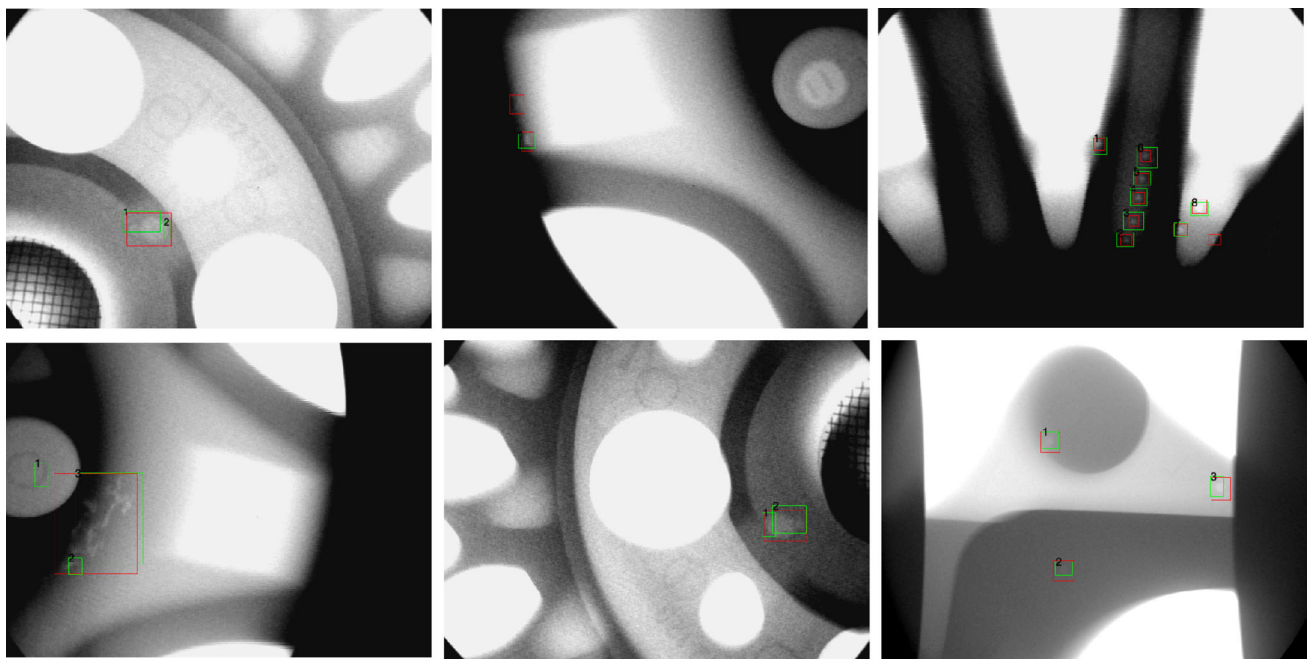


Fig. 10 Results using sliding-window approach with E100/G000 in series C0010, C0015, C0021, C0031, C0034 and C0061: detection in green, ground truth in red. For a result in series C0001 see Fig. 2

3.4 Discussion

In our experiments, we simulated two kinds of defects: simulated by an ellipsoidal model (ellipsoidal defects) and simulated by a GAN model (GAN defects). Both synthetic defects are very realistic as we can observe from Figs. 7 and 8 that show ellipsoidal and GAN defects respectively. With this result, it is clear that real defects, as shown in Fig. 3, are very similar to what can be seen by the naked eye.

The use of simulated defects in dataset plays a relevant role when training a CNN model for casting inspection. On the one hand, it is possible to have a large amount of data, that is necessary to train a CNN model adequately. On the other hand, it permits to have balanced classes. Using this data we could select the architecture of our CNN model. The selected CNN has five ‘conv’ blocks and ‘fc’ blocks. It has 30 layers with more than 1,350,000 parameters. This model achieved better results than other ones with other structures. We call this model Xnet-II. It has a better performance than our previous version Xnet [14] that has the structure shown in # 0 of Table 3.

In our results, it is clear that the use of GAN defects does not have an important effect on this problem. Indeed, when the training dataset consists of only GAN defects in the ‘defect’ class, the model is over-fitted on the training data (obtaining excellent accuracy in training and a poor accuracy in testing), probably because the generator is a sequential model, and our CNN can learn it easily (see row ‘E000/G100’ in Table 4 where training and validation accu-

racies are almost perfect, and mPA in sliding-window is the lowest of our experiments). Ellipsoidal defects seem to be very similar to real defects. The use of ellipsoidal defects in the training stage has an important effect on the performance on the testing data because CNN could learn a model that was able to distinguish them from the background.

As we can see in the results obtained by analyzing the whole X-ray images with sliding-windows, the effectiveness of the proposed has been validated. An important result is that the best performance has been achieved by training a general model, that we call ‘E100/ G000’, with data from several casting types. Thus, this unique general model is used to inspect casting from different types. This performance is better than that achieved by training an individual model for each casting type. Fig. 10 shows that the approach is able to detect real defects using a model that has been trained with simulated defects. The method, however, is far from perfect because there are some false positives (see green rectangles with no corresponding red ones) and false negatives (see red rectangles with no corresponding green ones).

The implementation of this model has been done in Python³ using Keras⁴ and TensorFlow⁵ libraries. Code and trained model will be available on our webpage after publication.

³ See <https://www.python.org>.

⁴ See <https://keras.io>.

⁵ See <https://www.tensorflow.org>.

4 Conclusions

In this work, we presented a Convolutional Neural Network (CNN) for defect detection in castings. In order to train the CNN model, a large dataset was necessary. The dataset could be built by using synthetic defects. They were simulated using 3D ellipsoidal models and Generative Adversarial Networks (GAN). In our experiments, the use of GAN defects does not play a relevant role in this solution. However, ellipsoidal defects helped to achieve better performance. Ellipsoidal defects from any size and orientation could be superimposed onto real X-ray images in any location. The proposed CNN architecture, with 30 layers and more than 1,350,000 was implemented in Python. It has been trained using a dataset with around 640,000 patches containing 50% of ellipsoidal defects and 50% of real background captured from different casting types. The model was tested using sliding-windows methodology on whole X-ray images achieving promising results ($mPA = 0.7102$). The model was able to detect real defects from different casting types.

In the future, we will train our deep learning network using a larger dataset with more casting types, and we will improve the sliding-windows strategy by including saliency maps and combining the detection windows with different scales. In addition, we will test other strategies like YOLO [42], ResNet [43] and multiple-view analysis [13].

It is worth noting, that this methodology can be applied to another kind of defects (such as discontinuities in the welding process [44], concrete defects [45], glass defects [46] or steel surface defects [47] among others) if we have a large enough database of representative images.

Acknowledgements This research was funded by Fondecyt Grant No. 1161314 from CONICYT, Chile.

References

- Mery, D.: *Computer Vision for X-Ray Testing*. Springer, New York (2015)
- Hanke, R., Fuchs, T., Uhlmann, N.: X-ray based methods for non-destructive testing and material characterization. *Nucl. Instrum. Methods Phys. Res. Sect. A* **591**(1), 14–18 (2008)
- Mery, D., Jaeger, T., Filbert, D.: A review of methods for automated recognition of casting defects. *Insight* **44**(7), 428–436 (2002)
- Mery, D.: Automated radioscopic testing of aluminum die castings. *Mater. Eval.* **64**(2), 135–143 (2006)
- Carrasco, M., Mery, D.: Automatic multiple view inspection using geometrical tracking and feature analysis in aluminum wheels. *Mach. Vis. Appl.* **22**(1), 157–170 (2011)
- Cogranne, R., Retraint, F.: Statistical detection of defects in radiographic images using an adaptive parametric model. *Signal Process.* **96**, 173–189 (2014)
- Ferguson, M., Ak, R., Lee, Y.T.T., Law, K.H.: Automatic localization of casting defects with convolutional neural networks. In: 2017 IEEE International Conference on Big Data (Big Data), pp. 1726–1735. IEEE (2017)
- Kamalakaran, A., Rajamanickam, G.: Spatial smoothing based segmentation method for internal defect detection in x-ray images of casting components. In: *Trends in Industrial Measurement and Automation (TIMA)*, vol. 2017, pp. 1–6. IEEE (2017)
- Li, X., Tso, S.K., Guan, X.P., Huang, Q.: Improving automatic detection of defects in castings by applying wavelet technique. *IEEE Trans. Ind. Electron.* **53**(6), 1927–1934 (2006)
- Li, W., Li, K., Huang, Y., Deng, X.: A new trend peak algorithm with X-ray image for wheel hubs detection and recognition. In: *Computational Intelligence and Intelligent Systems*, pp. 23–31. Springer (2015)
- Lin, J., Yao, Y., Ma, L., Wang, Y.: Detection of a casting defect tracked by deep convolution neural network. *Int. J. Adv. Manuf. Technol.* **97**(1–4), 573–581 (2018)
- Mery, D., Filbert, D.: Automated flaw detection in aluminum castings based on the tracking of potential defects in a radioscopic image sequence. *IEEE Trans. Robot. Autom.* **18**(6), 890–901 (2002)
- Mery, D.: Inspection of complex objects using multiple-X-ray views. *IEEE/ASME Trans. Mechatron.* **20**(1), 338–347 (2015)
- Mery, D., Arteta, C.: Automatic defect recognition in x-ray testing using computer vision. In: 2017 IEEE Winter Conference on Applications of Computer Vision (WACV), IEEE, pp. 1026–1035 (2017)
- Pieringer, C., Mery, D.: Flaw detection in aluminium die castings using simultaneous combination of multiple views. *Insight* **52**(10), 548–552 (2010)
- Pizarro, L., Mery, D., Delpiano, R., Carrasco, M.: Robust automated multiple view inspection. *Pattern Anal. Appl.* **11**(1), 21–32 (2008)
- Ramírez, F., Allende, H.: Detection of flaws in aluminium castings: a comparative study between generative and discriminant approaches. *Insight-Non-Destr. Test. Cond. Monit.* **55**(7), 366–371 (2013)
- Tang, Y., Zhang, X., Li, X., Guan, X.: Application of a new image segmentation method to detection of defects in castings. *Int. J. Adv. Manuf. Technol.* **43**(5–6), 431–439 (2009)
- Zhang, J., Guo, Z., Jiao, T., Wang, M.: Defect detection of aluminum alloy wheels in radiography images using adaptive threshold and morphological reconstruction. *Appl. Sci.* **8**(12), 2365 (2018)
- Zhao, X., He, Z., Zhang, S.: Defect detection of castings in radiography images using a robust statistical feature. *JOSA A* **31**(1), 196–205 (2014)
- Zhao, X., He, Z., Zhang, S., Liang, D.: A sparse-representation-based robust inspection system for hidden defects classification in casting components. *Neurocomputing* **153**, 1–10 (2015)
- LeCun, Y., Bengio, Y., Hinton, G.: Deep learning. *Nature* **521**(7553), 436–444 (2015)
- Bengio, Y., Courville, A., Vincent, P.: Representation learning: A review and new perspectives. *IEEE Trans. Pattern Anal. Mach. Intell.* **35**(8), 1798–1828 (2013)
- Simonyan, K., Zisserman, A.: Very deep convolutional networks for large-scale image recognition. *CoRR* [arXiv:1409.1556](https://arxiv.org/abs/1409.1556) (2014)
- LeCun, Y., Bottou, L., Bengio, Y., Haffner, P., et al.: Gradient-based learning applied to document recognition. *Proc. IEEE* **86**(11), 2278–2324 (1998)
- Krizhevsky, A., Sutskever, I., Hinton, G.E.: ImageNet classification with deep convolutional neural networks. In: *NIPS*, pp. 1106–1114 (2012)
- Russakovsky, O., Deng, J., Su, H., Krause, J., Satheesh, S., Ma, S., Huang, Z., Karpathy, A., Khosla, A., Bernstein, M., Berg, A., Fei-Fei, L.: ImageNet large scale visual recognition challenge. *Int. J. Comput. Vis.* **115**(3), 211–252 (2015)
- Goodfellow, I., Bengio, Y., Courville, A.: *Deep Learning*. The MIT Press, Cambridge (2016)

29. Nair, V., Hinton, G.E.: Rectified linear units improve restricted boltzmann machines. In: Proceedings of the 27th International Conference on Machine Learning (ICML-10), pp. 807–814 (2010)
30. Rumelhart, D.E., Hinton, G.E., Williams, R.J.: Learning representations by back-propagating errors. *Cogn. Model.* **5**(3), 1 (1998)
31. Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., Salakhutdinov, R.: Dropout: a simple way to prevent neural networks from overfitting. *J. Mach. Learn. Res.* **15**, 1929–1958 (2014)
32. Ioffe, S., Szegedy, C.: Batch normalization: accelerating deep network training by reducing internal covariate shift. *CoRR* [arXiv:1502.03167](https://arxiv.org/abs/1502.03167) (2015)
33. Carvajal, K., Chacon, M., Mery, D., Acuna, G.: Neural network method for failure detection with skewed class distribution. *Insight-Non-Destr. Test. Cond. Monit.* **46**(7), 399–402 (2004)
34. Mery, D.: A new algorithm for flaw simulation in castings by superimposing projections of 3d models onto x-ray images. In: 21st International Conference of the Chilean Computer Science Society (SCCC 2001), 6–8 Nov 2001, Punta Arenas, Chile, pp. 193–202 (2001)
35. Radford, A., Metz, L., Chintala, S.: Unsupervised representation learning with deep convolutional generative adversarial networks. *CoRR* [arXiv:1511.06434](https://arxiv.org/abs/1511.06434) (2015)
36. Martz, H., Logan, C., Schneberk, D., Shull, P.: *X-ray Imaging: Fundamentals, Industrial Techniques and Applications*. CRC Press, Boca Raton (2017)
37. Mery, D., Hahn, D., Hitschfeld, N.: Simulation of defects in aluminium castings using cad models of flaws and real x-ray images. *Insight-Non-Destr. Test. Cond. Monit.* **47**(10), 618–624 (2005). (Ron Halmshaw Award 2005)
38. Creswell, A., White, T., Dumoulin, V., Arulkumaran, K., Sengupta, B., Bharath, A.A.: Generative adversarial networks: an overview. *IEEE Signal Process. Mag.* **35**(1), 53–65 (2018)
39. Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., Bengio, Y.: Generative adversarial nets. In: *Advances in Neural Information Processing Systems*, pp. 2672–2680 (2014)
40. Mery, D., Riffo, V., Zscherpel, U., Mondragón, G., Lillo, I., Zuccar, I., Lobel, H., Carrasco, M.: GDxray: the database of X-ray images for nondestructive testing. *J. Nondestruct. Eval.* **34**(4), 1–12 (2015)
41. Everingham, M., Van Gool, L., Williams, C.K., Winn, J., Zisserman, A.: The pascal visual object classes (voc) challenge. *Int. J. Comput. Vis.* **88**(2), 303–338 (2010)
42. Redmon, J., Farhadi, A.: Yolov3: an incremental improvement. [arXiv:1804.02767](https://arxiv.org/abs/1804.02767) (2018)
43. He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 770–778 (2016)
44. Mery, D.: Automated detection of welding defects without segmentation. *Mater. Eval.* **69**(6), 657–663 (2011)
45. Gucunski, N., et al.: Nondestructive Testing to Identify Concrete Bridge Deck Deterioration. Transportation Research Board, Washington, DC (2013)
46. Carrasco, M., Pizarro, L., Mery, D.: Visual inspection of glass bottlenecks by multiple-view analysis. *Int. J. Comput. Integr. Manuf.* **23**(10), 925–941 (2010)
47. Liu, H.W., Lan, Y.Y., Lee, H.W., Liu, D.K.: Steel surface in-line inspection using machine vision. In: *First International Workshop on Pattern Recognition*, vol. 10011; 100110X. International Society for Optics and Photonics (2016)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Terms and Conditions

Springer Nature journal content, brought to you courtesy of Springer Nature Customer Service Center GmbH (“Springer Nature”).

Springer Nature supports a reasonable amount of sharing of research papers by authors, subscribers and authorised users (“Users”), for small-scale personal, non-commercial use provided that all copyright, trade and service marks and other proprietary notices are maintained. By accessing, sharing, receiving or otherwise using the Springer Nature journal content you agree to these terms of use (“Terms”). For these purposes, Springer Nature considers academic use (by researchers and students) to be non-commercial.

These Terms are supplementary and will apply in addition to any applicable website terms and conditions, a relevant site licence or a personal subscription. These Terms will prevail over any conflict or ambiguity with regards to the relevant terms, a site licence or a personal subscription (to the extent of the conflict or ambiguity only). For Creative Commons-licensed articles, the terms of the Creative Commons license used will apply.

We collect and use personal data to provide access to the Springer Nature journal content. We may also use these personal data internally within ResearchGate and Springer Nature and as agreed share it, in an anonymised way, for purposes of tracking, analysis and reporting. We will not otherwise disclose your personal data outside the ResearchGate or the Springer Nature group of companies unless we have your permission as detailed in the Privacy Policy.

While Users may use the Springer Nature journal content for small scale, personal non-commercial use, it is important to note that Users may not:

1. use such content for the purpose of providing other users with access on a regular or large scale basis or as a means to circumvent access control;
2. use such content where to do so would be considered a criminal or statutory offence in any jurisdiction, or gives rise to civil liability, or is otherwise unlawful;
3. falsely or misleadingly imply or suggest endorsement, approval, sponsorship, or association unless explicitly agreed to by Springer Nature in writing;
4. use bots or other automated methods to access the content or redirect messages
5. override any security feature or exclusionary protocol; or
6. share the content in order to create substitute for Springer Nature products or services or a systematic database of Springer Nature journal content.

In line with the restriction against commercial use, Springer Nature does not permit the creation of a product or service that creates revenue, royalties, rent or income from our content or its inclusion as part of a paid for service or for other commercial gain. Springer Nature journal content cannot be used for inter-library loans and librarians may not upload Springer Nature journal content on a large scale into their, or any other, institutional repository.

These terms of use are reviewed regularly and may be amended at any time. Springer Nature is not obligated to publish any information or content on this website and may remove it or features or functionality at our sole discretion, at any time with or without notice. Springer Nature may revoke this licence to you at any time and remove access to any copies of the Springer Nature journal content which have been saved.

To the fullest extent permitted by law, Springer Nature makes no warranties, representations or guarantees to Users, either express or implied with respect to the Springer nature journal content and all parties disclaim and waive any implied warranties or warranties imposed by law, including merchantability or fitness for any particular purpose.

Please note that these rights do not automatically extend to content, data or other material published by Springer Nature that may be licensed from third parties.

If you would like to use or distribute our Springer Nature journal content to a wider audience or on a regular basis or in any other manner not expressly permitted by these Terms, please contact Springer Nature at

onlineservice@springernature.com